PROJECT DOCUMENTATION

South African E-Commerce Data Pipeline

A production-shaped Extract → Validate → Transform → Load → Analyse pipeline for retail inventory analytics.

Repository	`git@github.com:Lebo-Moropene/sa-ecommerce-data-pipeline.git`
Stack	Python 3.10+, `requests`, `pandas`, SQLite, SQL, `pytest`
Source	`dummyjson.com/products` (public sample REST API)
Status	Complete — all four pipeline stages implemented, tested and verified end to end.

This document describes what the pipeline is, the problem it solves, how it is architected, and how to install, run, extend and troubleshoot it. It was generated from the repository source at commit `82a4721`.

Contents

1 Executive Summary **3**
1.1 At a glance . . . 3

2 The Problem This Pipeline Solves **3**
2.1 The business problem . . . 3
2.2 The technical problem . . . 4
2.3 Problem to solution . . . 4

3 How It Works **4**
3.1 Architecture at a glance . . . 4
3.2 Stage 1 — Extract . . . 5
3.3 Stage 2 — Validate and Transform . . . 5
3.4 Stage 3 — Load . . . 6
3.5 Stage 4 — Analyse . . . 6
3.6 Orchestration . . . 6

4 Technology Stack **7**

5 Repository Layout **8**
5.1 How the layout is organised . . . 8
5.2 What is not in version control . . . 8

6 Getting Started **8**
6.1 Prerequisites . . . 9
6.2 Clone and set up . . . 9
6.3 Verify the installation . . . 9

7 Usage Guide **9**
7.1 Run the whole pipeline . . . 9
7.2 Run a single stage . . . 10
7.3 Inspect the intermediate artefacts . . . 10
7.4 Run the tests . . . 11
7.5 Query the database yourself . . . 11

8 Data Model and Artefacts **11**
8.1 Files produced by a run . . . 11
8.2 The products table . . . 12
8.3 Data types and coercion . . . 12

9 Data Quality: Validation Rules **12**
9.1 The rule set . . . 12
9.2 What is deliberately *not* rejected . . . 13
9.3 Schema failures versus row failures . . . 13
9.4 Reading the reject file . . . 13

10 The Three Analytics Reports **14**
10.1 Report 1 — Products with the lowest stock . . . 14
10.2 Report 2 — Products per category . . . 14
10.3 Report 3 — Average price per category . . . 14
10.4 Why the reports live in SQL . . . 15

11 Testing **15**
11.1 The suite . . . 15
11.2 The fixture . . . 15

12 Design Decisions **17**

13 Troubleshooting **18**

14 Limitations and Roadmap **18**
14.1 Current limitations 18
14.2 Roadmap 19

A Command Reference **19**

B References **19**

1 Executive Summary

This project is a complete, working Extract–Transform–Load–Analyse (ETLA) data engineering pipeline written in Python. It pulls a live product catalogue from a public REST API, puts that data through a deliberate validation and cleaning stage, loads the survivors into a SQLite database, and finally runs three inventory and category analytics reports against the result.

The pipeline exists to answer three questions that every online retailer has to answer continuously and correctly:

1. **What is about to run out?** Which products have the lowest stock on hand and need replenishment first.
2. **What does the catalogue look like?** How the assortment is distributed across categories, which tells you about shelf space, merchandising effort and marketing focus.
3. **How is the catalogue priced?** The average price point per category, which tells you about price positioning and leaves room for promotional strategy.

What separates this from a script that calls an API and prints a table is the middle of the pipeline. Data arriving from an API is untrusted: fields are missing, numbers arrive as strings, titles are blank, and identifiers repeat. The pipeline treats that as the normal case rather than the exception. Every record is checked against an explicit rule set, and anything that fails is routed to a *reject file* with the reason implied by the rule it broke. Nothing is silently dropped, and nothing unvalidated ever reaches the database.

The project is deliberately small and dependency-light — four runtime packages, one file per pipeline stage, and a SQLite file you can open with the `sqlite3` command-line tool on any machine — which makes it easy to read end to end, easy to demonstrate, and easy to extend with more sources or more reports.

1.1 At a glance

Language	Python 3.10 or newer
Pipeline stages	Extract → Validate/Transform → Load → Analyse
Runtime deps	`requests`, `pandas`
Test dependency	`pytest`
Data store	SQLite (single file, `data/ecommerce.db`)
Analytics	3 reports, written in SQL and executed in the database layer
Tests	3 automated tests covering the three report queries
Entry point	`python-msrc.pipeline`
Lines of Python	5 modules, 4 pipeline stages plus the orchestrator
Repository	`git@github.com:Lebo-Moropene/sa-ecommerce-data-pipeline.git`

2 The Problem This Pipeline Solves

2.1 The business problem

South African e-commerce operators face the same inventory pressure as any competitive online retailer, and the pressure shows up in three concrete places.

Stockouts. A product that is out of stock does not generate a slow sale — it generates no sale at all, and it quietly pushes the customer to a competitor. The catalogue in this pipeline contains a fragrance with only four units left. Nothing about that record looks urgent in a raw JSON feed. It only becomes urgent once it is ranked, aggregated and presented next to everything else that is low.

Blind spots in the assortment. A buyer or merchandiser needs to know whether the catalogue is well balanced or lopsided. If five hundred of the products are in one category, the category is the store and the other categories are shelf dressing. A simple count per category surfaces that immediately — but only if the data is clean enough to count.

Price positioning. Average price per category is the cheapest way to see whether the assortment sits in the premium or the budget tier, and therefore where promotions, bundles or supplier negotiations will pay off.

2.2 The technical problem

The business questions are easy to state and awkward to answer with dirty data. A live product API returns records with missing brands, prices encoded as strings, blank titles, duplicated identifiers and negative stock levels. Any analysis written directly against that payload produces confident, well-formatted, quietly wrong answers — because the aggregation ran over rows that should never have been counted.

This is the failure mode the pipeline is engineered to prevent. The validation stage sits between extraction and analysis specifically so that the reports operate on a dataset whose contents are known and auditable.

2.3 Problem to solution

Problem	Why it hurts	How this pipeline handles it
Stock running low is invisible in a raw feed	Replenishment happens after the stockout, not before it	Ranks products by stock ascending and prints the five closest to running out
Catalogue is skewed by category	Merchandising and marketing effort is misallocated	Counts products per category and orders by volume
Price positioning is unknown	Promotions are priced by gut feel	Averages price per category and orders by price point
API records are incomplete or invalid	Reports quietly aggregate nonsense	Twelve validation rules reject bad rows to an auditable CSV
Re-running the pipeline duplicates rows	Counts inflate on every run	The load step replaces the table rather than appending to it
The pipeline only works from one directory	Breaks the moment it is scheduled from elsewhere	Every path is resolved relative to the source file, not the shell's CWD

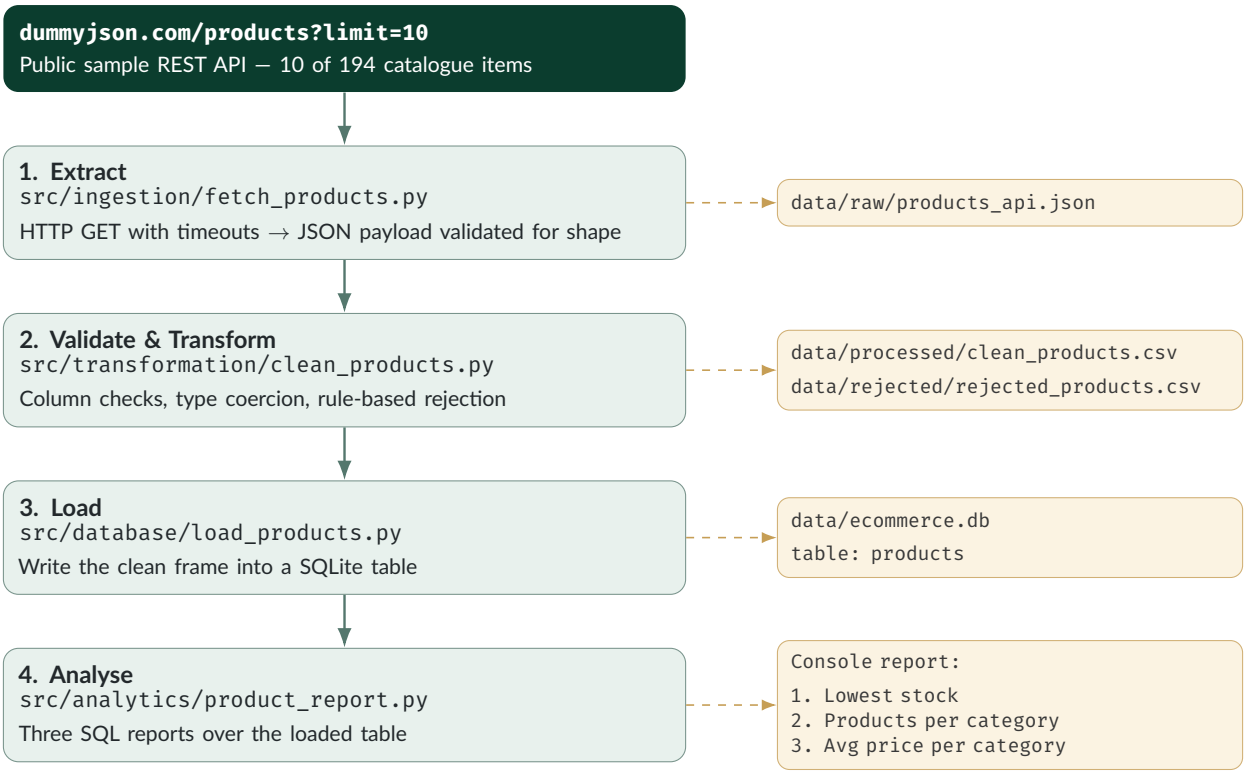
The core design decision

Data quality is enforced *before* analysis, not alongside it. A rejected record never reaches the database, so it can never be counted, averaged or ranked. Every rejection is written to `data/rejected/rejected_products.csv` so that the volume and nature of bad data is itself a reportable signal.

3 How It Works

3.1 Architecture at a glance

The pipeline is a linear sequence of four stages. Each stage is a standalone module with a single public function, reads from one location and writes to another, and can be executed on its own — which is what makes the pipeline debuggable: you can re-run just the stage that broke and keep the results of the ones before it.



3.2 Stage 1 — Extract

`src/ingestion/fetch_products.py` issues a single HTTP GET against the catalogue endpoint with a limit of ten records.

```
API_URL = "https://dummyjson.com/products?limit=10"

response = requests.get(API_URL, timeout=(5, 15))
response.raise_for_status()
payload = response.json()
```

The response is written verbatim to `data/raw/products_api.json` before any interpretation happens. Keeping the untouched payload on disk means the extraction stage is reproducible: if a later stage produces a surprising result, the exact bytes that produced it are still there.

The function is defensive about everything it does not control. Network failures, HTTP error statuses and malformed JSON are all caught and re-raised as a `SystemExit` with a readable message rather than a traceback; and the decoded JSON is checked to be an object containing a list of products before it is trusted.

Failure modes

- `requests.RequestException` → *HTTP request failed: …*
- Non-JSON body → *API returned invalid JSON: …*
- Payload that is not an object, or has no `products` list → *Unexpected API response: …*

3.3 Stage 2 — Validate and Transform

`src/transformation/clean_products.py` loads the raw payload into a `pandas` DataFrame and applies the rule set described in Section 9. Records that satisfy every rule become the clean dataset; the rest are written to the reject file.

```
required_columns = ["id", "title", "category", "price", "stock"]
for column in required_columns:
    if column not in df.columns:
        raise ValueError(f"Required column is missing: {column}")

df["price"] = pd.to_numeric(df["price"], errors="coerce")
df["stock"] = pd.to_numeric(df["stock"], errors="coerce")

invalid_rows = (
    df["id"].isna()
    | df["title"].isna()
    | df["title"].astype(str).str.strip().eq("")
    | df["category"].isna()
    | df["price"].isna()
    | df["stock"].isna()
    | (df["price"] < 0)
    | (df["stock"] < 0)
    | df["id"].duplicated(keep="first")
)

rejected = df[invalid_rows].copy()
clean = df[~invalid_rows].copy()
```

Two details are worth calling out. First, `errors="coerce"` is what makes the numeric checks reliable: a price of `"notanumber"` becomes `NaN` rather than raising, and the existing `isna()` test then rejects it. The pipeline coerces first and judges second, so no malformed value can leak past a comparison. Second, the duplicate check uses `duplicated(keep="first")`, which retains the first occurrence and rejects only the repeats — the opposite of the naive `keep="last"` behaviour, which would silently prefer the newer record.

3.4 Stage 3 — Load

`src/database/load_products.py` writes the clean frame into a SQLite database using `pandas`' `to_sql` helper.

```
df = pd.read_csv(CSV_PATH)

with sqlite3.connect(DATABASE_PATH) as connection:
    df.to_sql("products", connection, if_exists="replace", index=False)

    count = connection.execute(
        "SELECT COUNT(*) FROM products"
    ).fetchone()[0]
```

`if_exists="replace"` is what makes the pipeline idempotent. Every run rebuilds the `products` table from the current clean file, so running the pipeline ten times gives the same ten rows — not a hundred. The stage also refuses to invent data: if the clean CSV does not exist it raises `FileNotFoundError` with a message telling you to run the cleaning step first, rather than quietly loading an empty table.

3.5 Stage 4 — Analyse

`src/analytics/product_report.py` runs three queries against the loaded database and prints the results. The analysis lives in SQL, executed by SQLite, rather than in Python — the database does the aggregation, and the Python layer only formats the output. Section 10 covers each report in detail.

3.6 Orchestration

`src/pipeline.py` chains the four stages together. It is deliberately thin: four calls in order, a banner before each step, and a completion message at the end.

```
def main():
    print("\nSTEP 1: Fetching product data")
    fetch_products()

    print("\nSTEP 2: Cleaning product data")
    clean_products()

    print("\nSTEP 3: Loading products into SQLite")
    load_products()

    print("\nSTEP 4: Running analytics")
    run_report()

    print("\nPipeline completed successfully.")
```

Because every stage is an ordinary importable function with no shared state, the orchestrator is a convenience, not a dependency. Running `python-msrc.ingestion.fetch_products` by hand produces exactly the same artefact as running the full pipeline.

4 Technology Stack

Layer	Tool	Why it was chosen
Language	Python 3.10+	The dominant language for data work; readable for anyone who has not written Python before, and the language the source API ecosystem is built on.
Ingestion	`requests`	A mature HTTP client with first-class timeout support and clear exception types, which is what makes the error handling in Stage 1 straightforward.
Transformation	`pandas`	Vectorised column operations turn eleven validation rules into a single boolean mask, and the DataFrame is a natural fit for reading and writing CSV.
Storage	SQLite	A single file, zero server setup, and a full SQL engine. The database is easy to inspect with the `sqlite3` CLI, which makes the analysis stage transparent rather than a black box.
Analytics	SQL	Aggregation stays in the database where it belongs, and the three reports are short enough to read in full.
Testing	`pytest`	Minimal fixtures, plain asserts, and a test suite that runs in under a tenth of a second.
Version control	Git / GitHub	Branch-per-feature history, and a remote that anyone can clone and run.

The whole runtime footprint is `requests`, `pandas` and the Python standard library. There is no ORM, no framework, no scheduler, no configuration file and no secrets to manage. For a pipeline this size that is a feature: there is nothing to learn before you can run it.

5 Repository Layout

```
.
|-- .gitignore                # excludes venv, caches and generated data
|-- README.md                 # quick-start guide
|-- requirements.txt          # requests, pandas, pytest
|
|-- data/
|   |-- raw/                  # API responses          (gitignored)
|   |-- processed/            # cleaned CSV output      (gitignored)
|   |-- rejected/             # rows that failed checks (gitignored)
|   `-- ecommerce.db          # SQLite database         (gitignored)
|
|-- docs/                     # project documentation
|-- logs/                     # pipeline logs           (gitignored)
|-- sql/                      # ad-hoc SQL queries
|
|-- src/
|   |-- __init__.py
|   |-- pipeline.py           # end-to-end orchestrator
|   |-- ingestion/
|   |   |-- __init__.py
|   |   `-- fetch_products.py     # Stage 1
|   |-- transformation/
|   |   |-- __init__.py
|   |   `-- clean_products.py     # Stage 2
|   |-- database/
|   |   |-- __init__.py
|   |   `-- load_products.py      # Stage 3
|   `-- analytics/
|       |-- __init__.py
|       `-- product_report.py     # Stage 4
|
`-- tests/
    `-- test_product_report.py    # pytest suite
```

5.1 How the layout is organised

The `src` tree mirrors the pipeline itself: one package per stage, one module per stage, named for what it does rather than for a technical layer. Each package has an `__init__.py` so that it is importable, and each module exposes exactly one public function with the same name as its purpose — `fetch_products`, `clean_products`, `load_products`, `run_report`.

The `data` tree separates artefacts by their position in the pipeline rather than by file type: everything the API produced lives in `raw`, everything that passed validation lives in `processed`, and everything that failed lives in `rejected`. When you open a directory you know immediately which stage produced the file and what its status is.

5.2 What is not in version control

`.gitignore` excludes the virtual environment, Python bytecode caches, the pytest cache, IDE settings and — importantly — all generated data files. Only the code, the documentation and empty-directory placeholders are committed. This keeps the repository small and reviewable, and it means a fresh clone is a clean slate: the `data` directories exist, ready to be filled by the first pipeline run.

6 Getting Started

6.1 Prerequisites

Requirement	Details
Python 3.10 or newer	Check with `python3--version`.
Git	Needed only to clone the repository.
Network access	The pipeline calls a live public API.
`sqlite3` CLI	Optional — only for ad-hoc queries in Section 7.5.

6.2 Clone and set up

```
git clone git@github.com:Lebo-Moropene/sa-ecommerce-data-pipeline.git
cd sa-ecommerce-data-pipeline

python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

The virtual environment keeps the three dependencies isolated from the system Python, and it is the reason `.venv/` is the first entry in `.gitignore`. On Windows, activate with `.venv\TU\textbackslashScripts\TU\textbackslashactivate` instead.

6.3 Verify the installation

Before running the pipeline, confirm the dependencies resolved correctly:

```
python -m pytest tests/ -v
```

```
platform linux -- Python 3.10.12, pytest-9.1.1
collected 3 items

tests/test_product_report.py::test_products_are_counted_by_category PASSED [ 33%]
tests/test_product_report.py::test_lowest_stock_product_is_returned_first PASSED [ 66%]
tests/test_product_report.py::test_average_price_is_calculated_correctly PASSED [100%]

============================== 3 passed in 0.02s ===============================
```

A green suite here means the environment is sound and the report queries behave as expected — before you add network variables into the picture.

7 Usage Guide

7.1 Run the whole pipeline

```
python -m src.pipeline
```

This is the single command that matters. It runs all four stages in order and prints the real output of the run — the console report in Section 10 is the literal output of the command below, captured from a live run against the public API.

```
STEP 1: Fetching product data
HTTP status: 200
Products retrieved: 10
Total products reported by API: 194
Raw data saved to: .../data/raw/products_api.json

STEP 2: Cleaning product data
Products received: 10
Clean products: 10
Rejected products: 0
Clean data saved to: .../data/processed/clean_products.csv
Rejected data saved to: .../data/rejected/rejected_products.csv

STEP 3: Loading products into SQLite
Products loaded into database: 10
Database saved to: .../data/ecommerce.db

STEP 4: Running analytics

=== 1. PRODUCTS WITH LOWEST STOCK ===
Dolce Shine Eau de | fragrances | Stock: 4
Calvin Klein CK One | fragrances | Stock: 29
Eyeshadow Palette with Mirror | beauty | Stock: 34
Chanel Coco Noir Eau De | fragrances | Stock: 58
Red Nail Polish | beauty | Stock: 79

=== 2. PRODUCTS PER CATEGORY ===
beauty: 5 products
fragrances: 5 products

=== 3. AVERAGE PRICE PER CATEGORY ===
fragrances: 83.99
beauty: 13.39

Pipeline completed successfully.
```

Two numbers in that output are worth pausing on. `Productsretrieved:10` is what the API returned, limited by the request. `TotalproductsreportedbyAPI:194` is the size of the full catalogue the API knows about. The pipeline deliberately fetches a small slice — the point of the project is the architecture, not the volume of data, and a ten-row payload keeps every stage and every report readable.

7.2 Run a single stage

Each stage is a standalone module. This is the primary debugging technique: if the report looks wrong, re-run just the stage that produced its input and inspect the intermediate file.

```
python -m src.ingestion.fetch_products    # Stage 1 -> data/raw/
python -m src.transformation.clean_products   # Stage 2 -> data/processed/, data/rejected/
python -m src.database.load_products     # Stage 3 -> data/ecommerce.db
python -m src.analytics.product_report  # Stage 4 -> console report
```

7.3 Inspect the intermediate artefacts

```
# the ten rows that survived validation
cat data/processed/clean_products.csv

# anything that failed a rule (empty when the source is clean)
cat data/rejected/rejected_products.csv

# the loaded table
sqlite3 data/ecommerce.db ".tables"
sqlite3 data/ecommerce.db "SELECT COUNT(*) FROM products;"
```

```
id,title,category,brand,price,stock,rating
1,Essence Mascara Lash Princess,beauty,Essence,9.99,99,2.56
2,Eyeshadow Palette with Mirror,beauty,Glamour Beauty,19.99,34,2.86
3,Powder Canister,beauty,Velvet Touch,14.99,89,4.64
4,Red Lipstick,beauty,Chic Cosmetics,12.99,91,4.36
5,Red Nail Polish,beauty,Nail Couture,8.99,79,4.32
6,Calvin Klein CK One,fragrances,Calvin Klein,49.99,29,4.37
7,Chanel Coco Noir Eau De,fragrances,Chanel,129.99,58,4.26
8,Dior J'adore,fragrances,Dior,89.99,98,3.8
9,Dolce Shine Eau de,fragrances,Dolce & Gabbana,69.99,4,3.96
10,Gucci Bloom Eau de,fragrances,Gucci,79.99,91,2.74
```

7.4 Run the tests

```
python -m pytest tests/ -v      # verbose
python -m pytest tests/ -q      # quiet
python -m pytest tests/ --tb=short   # shorter tracebacks on failure
```

7.5 Query the database yourself

Because the data lands in a real SQLite file, you are not limited to the three built-in reports. Open the database and ask it anything.

```
sqlite3 data/ecommerce.db "SELECT category, AVG(price) FROM products GROUP BY category;"

sqlite3 data/ecommerce.db "SELECT title, stock FROM products WHERE stock < 50 ORDER BY stock;"

sqlite3 data/ecommerce.db "SELECT brand, COUNT(*) FROM products GROUP BY brand ORDER BY 2 DESC;"

sqlite3 data/ecommerce.db ".schema products"
```

```
fragrances|83.9925
beauty|13.39
```

8 Data Model and Artefacts

8.1 Files produced by a run

Artefact	Produced by	Contents
`data/raw/products_api.json`	Stage 1	The verbatim API payload, including metadata such as the catalogue total.
`data/processed/clean_products.csv`	Stage 2	Every record that passed all validation rules, in the canonical column order.
`data/rejected/rejected_products.csv`	Stage 2	Every record that failed a rule, with the full original row so the failure can be diagnosed.
`data/ecommerce.db`	Stage 3	A SQLite database containing the `products` table.

The clean CSV keeps exactly seven columns, in a fixed order: `id`, `title`, `category`, `brand`, `price`, `stock`, `rating`. Everything else the API sends — descriptions, discount percentages, SKU, weight, dimensions, warranty information, shipping information, availability status, reviews, return policy, minimum order quantity, metadata and image URLs — is discarded. That is intentional: the pipeline stores what the analytics need and the business

asks about, rather than a copy of everything the API happens to return.

8.2 The products table

The schema is inferred by `pandas` from the clean CSV, so it follows the data rather than being declared separately. The table is called `products`:

```
CREATE TABLE "products" (
"id"       INTEGER,
"title"    TEXT,
"category" TEXT,
"brand"    TEXT,
"price"    REAL,
"stock"    INTEGER,
"rating"   REAL
)
```

Column	Type	Meaning and rules
`id`	INTEGER	Product identifier from the API. Unique — the first occurrence wins, later duplicates are rejected.
`title`	TEXT	Display name. Never null and never blank after stripping whitespace.
`category`	TEXT	Catalogue grouping, used by reports 2 and 3.
`brand`	TEXT	Manufacturer. Missing values are filled with the literal `Unknown` rather than being rejected.
`price`	REAL	Unit price. Numeric, non-negative. Used by report 3.
`stock`	INTEGER	Units on hand. Numeric, non-negative. Used by report 1.
`rating`	REAL	Customer rating. Optional — a missing rating does not cause a rejection.

8.3 Data types and coercion

`price` and `stock` arrive from the API as whatever the producer chose to send. The pipeline coerces both with `pd.to_numeric(...,errors="coerce")` before any validation, which produces exactly the behaviour you want: a well-formed number stays a number, and anything unparseable becomes `NaN` and is then rejected by the standard missing-value rule. There is no third path where a malformed value survives as a string.

9 Data Quality: Validation Rules

9.1 The rule set

A record is written to the reject file if it breaks any of the following.

#	Rule	Why it matters
1	`id` is missing	The identifier is the key; a record without it cannot be deduplicated or referenced.
2	`title` is missing	A product with no name cannot be presented to a buyer or a buyer-facing report.
3	`title` is blank or whitespace-only	Semantically identical to a missing title, but survives an `isna()` check, so it needs its own rule.
4	`category` is missing	Category drives two of the three reports.
5	`price` is missing or not numeric	Unparseable prices become `NaN` during coercion and are caught here.
6	`stock` is missing or not numeric	The same, for the field that drives the replenishment report.
7	`price` is negative	A negative price is a sign error or a malformed discount, not a real price.
8	`stock` is negative	Negative stock is always a data defect and would corrupt the low-stock ranking.
9	`id` is a duplicate	Duplicate rows would double-count every aggregate that touches them.

Rules 5 and 6 in the table cover the numeric-coercion failures described in Section 8.3: a value that cannot be parsed as a number is caught by the same missing-value test as a value that was never supplied.

9.2 What is deliberately *not* rejected

- **A missing `brand`.** Replaced with `Unknown` and kept. The brand does not drive any of the three reports, so a blank manufacturer is not a reason to lose a saleable product.
- **A missing `rating`.** Kept as null. The field is descriptive rather than analytical.
- **Extra columns.** Fields the pipeline does not model are dropped silently — they are not part of the contract, not defects.

9.3 Schema failures versus row failures

These are handled at two different levels, and the distinction matters.

A **row failure** is a record that breaks a rule. It is expected, it is recorded in the reject file, and the pipeline continues. This is normal operation.

A **schema failure** is a required column missing from the payload entirely — not a row with a bad value, but a structural change in the API. In that case the pipeline raises `ValueError` and stops. There is no point in producing a clean dataset from a payload that no longer has the shape the pipeline depends on; the correct response is to stop and let a human decide what changed.

```
required_columns = ["id", "title", "category", "price", "stock"]
for column in required_columns:
    if column not in df.columns:
        raise ValueError(f"Required column is missing: {column}")
```

9.4 Reading the reject file

The reject file contains the *original* rows — every column the API sent — not a list of reasons. That choice keeps the file a faithful record of what arrived, and it means you can inspect, group and count failures without the pipeline having to maintain a parallel error schema.

```
$ wc -l data/rejected/rejected_products.csv
1 data/rejected/rejected_products.csv

$ head -2 data/rejected/rejected_products.csv
```

```
id,title,description,category,price,discountPercentage,rating,stock,...
```

A rejection rate that climbs from zero is a signal worth acting on: it usually means the upstream API changed its contract, and the pipeline is telling you before the reports quietly go wrong.

10 The Three Analytics Reports

All three reports run in `src/analytics/product_report.py` against the loaded `products` table. The aggregation is performed by SQLite; the Python layer only iterates over the result and prints it.

10.1 Report 1 — Products with the lowest stock

```
SELECT title, category, stock
FROM products
ORDER BY stock ASC
LIMIT 5
```

The five products closest to running out, ascending by stock. This is the replenishment list: in a live catalogue run it is immediately actionable, because it identifies both the product and the quantity remaining.

```
=== 1. PRODUCTS WITH LOWEST STOCK ===
Dolce Shine Eau de | fragrances | Stock: 4
Calvin Klein CK One | fragrances | Stock: 29
Eyeshadow Palette with Mirror | beauty | Stock: 34
Chanel Coco Noir Eau De | fragrances | Stock: 58
Red Nail Polish | beauty | Stock: 79
```

10.2 Report 2 — Products per category

```
SELECT category, COUNT(*) AS product_count
FROM products
GROUP BY category
ORDER BY product_count DESC, category ASC
```

The catalogue mix, largest category first. The secondary sort on category name makes the output deterministic when two categories tie — without it, the order of tied rows would depend on SQLite's internal execution plan and could differ between runs.

```
=== 2. PRODUCTS PER CATEGORY ===
beauty: 5 products
fragrances: 5 products
```

10.3 Report 3 — Average price per category

```
SELECT category, ROUND(AVG(price), 2) AS average_price
FROM products
GROUP BY category
ORDER BY average_price DESC
```

Price positioning by category, most expensive first. Rounding in SQL rather than in Python keeps the arithmetic in one place, and the `:.2f` format specifier guarantees two decimal places in the printed output regardless of the underlying value.

```
=== 3. AVERAGE PRICE PER CATEGORY ===
fragrances: 83.99
beauty: 13.39
```

10.4 Why the reports live in SQL

Putting the aggregation in the database rather than in Python has three practical benefits:

- **Correctness by construction.** SQLite applies the `NULL` semantics of `AVG` and `COUNT` correctly without you having to reason about Python's treatment of missing values in aggregates.
- **Readability.** Each report is four or five lines of obvious SQL. You can paste any of them into the `sqlite3` CLI and get the same answer — which is exactly what Section 7.5 invites you to do.
- **Extensibility.** Adding a report means adding a query, not threading a new data structure through the Python layer.

11 Testing

11.1 The suite

The project ships three tests in `tests/test_product_report.py`, covering the three report queries.

Test	What it asserts
`test_products_are_counted_by_category`	Grouping by category returns the correct counts for each group.
`test_lowest_stock_product_is_returned_first`	Ordering by stock ascending and limiting to one returns the lowest-stock product.
`test_average_price_is_calculated_correctly`	The average price for the `beauty` category equals 15.0, using `pytest.approx` for floating-point comparison.

11.2 The fixture

All three tests share a single `database` fixture that builds an in-memory SQLite database with three rows spanning two categories.

```
@pytest.fixture
def database():
    connection = sqlite3.connect(":memory:")

    connection.execute("""
        CREATE TABLE products (
            id INTEGER PRIMARY KEY,
            title TEXT NOT NULL,
            category TEXT NOT NULL,
            price REAL NOT NULL,
            stock INTEGER NOT NULL
        )
    """)

    connection.executemany(
        """
        INSERT INTO products (id, title, category, price, stock)
        VALUES (?, ?, ?, ?, ?)
        """,
```

```
        [
            (1, "Product A", "beauty", 10.0, 5),
            (2, "Product B", "beauty", 20.0, 15),
            (3, "Product C", "fragrances", 60.0, 2),
        ],
    )

    yield connection
    connection.close()
```

Using `:memory:` means the tests never touch `data/ecommerce.db`, never depend on a previous pipeline run, and cannot leave state behind. They are also fast enough to run on every save.

A deliberate trade-off

The tests exercise the *queries* rather than importing `run_report` from `src/analytics/product_report.py`. That keeps the suite independent of the production module and of the file system, but it also means the SQL is duplicated: if the query inside the report module changed, these tests would still pass. Moving the queries into shared constants that both the module and the tests import would close that gap. It is listed in Section 14.2.

12 Design Decisions

Decision	Rationale
Paths resolved from `__file__`	Every module computes its paths with `Path(__file__).resolve().parents[2]`, which points at the repository root. The pipeline therefore behaves identically whether it is run from the project root, from `src/`, or from a cron job in `/`.
CSV between stages	A CSV intermediate is inspectable with `cat`, diffable between runs, and readable in any spreadsheet. It also decouples the stages: changing the load step does not require re-running the API call.
Reject file instead of silent drops	Discarding bad rows without recording them makes data loss invisible. Writing them out makes the pipeline auditable and turns the rejection count into a metric in its own right.
Coerce, then validate	Converting types before applying rules means no malformed value can survive a comparison. The alternative — validating strings — requires two versions of every rule.
`if_exists="replace"` on load	Makes the pipeline idempotent. Repeated runs produce identical tables instead of accumulating duplicates.
Timeouts on every request	`timeout=(5,15)` sets a five-second connect timeout and a fifteen-second read timeout. Without them a stalled server would hang the pipeline indefinitely, which is the difference between a failed job and a stuck one.
`SystemExit` for expected failures	Network and schema errors are not bugs, they are operational realities. Raising `SystemExit` with a one-line message gives the operator a readable error and a non-zero exit code, instead of a stack trace.
SQL for the analytics	Aggregation belongs in the database. It is faster, it is correct about `NULL`, and the queries stay short enough to audit by eye.
SQLite over a client/server database	A single file means no server to install, no credentials to manage and no network hop between the pipeline and its own data — while still providing a complete SQL engine.
Generated data kept out of Git	Committing the database and the CSVs would bloat every commit with regenerable content and create merge conflicts on data that no human edits.
No configuration file	The API URL and the record limit are module constants. For a pipeline with one source and one limit, a configuration layer would add a file to read and an argument to understand, in exchange for nothing.

13 Troubleshooting

Symptom	Likely cause	Fix
`ModuleNotFoundError: Nomodulenamed'requests'`	The virtual environment is not active, or dependencies were not installed	Run `source.venv/bin/activate` then `pipinstall-rrequirements.txt`
`HTTPrequestfailed: \protect\TU\ textellipsis`	No network access, DNS failure, or the API is unreachable	Check connectivity, then re-run; the timeout is 5s connect / 15s read
`APIreturnedinvalidJSON: \protect\TU\ textellipsis`	The endpoint returned an error page instead of JSON	Re-run; if it persists the API contract has probably changed
`FileNotFoundError: CleanCSVnotfound`	Stage 3 was run before Stage 2	Run `python-msrc.transformation. clean_products` first
`Requiredcolumnismissing: price`	The API response no longer has the expected structure	Inspect `data/raw/products_api.json` and update the column list
Reports show zero rows	The clean CSV was empty because every record was rejected	Check `data/rejected/rejected_ products.csv` for the cause
Counts are larger than expected	A pipeline stage was run twice against a table that appends	Not possible with the current loader — `if_exists="replace"` always rebuilds the table
`sqlite3:commandnotfound`	The SQLite CLI is not installed	Install it, or query the database from Python with the `sqlite3` module
Tests fail after editing a query	The suite duplicates the report SQL rather than importing it	Update the SQL in both `tests/test_product_report.py` and `src/analytics/product_report.py`

14 Limitations and Roadmap

14.1 Current limitations

- **A ten-record slice.** The pipeline fetches `limit=10` of a 194-record catalogue. This keeps the output readable, but means the reports describe a sample rather than the whole assortment.
- **No pagination.** Extending to the full catalogue requires either a larger limit or a loop over pages, which the ingestion stage does not implement.
- **Single-file, single-writer storage.** SQLite handles this workload comfortably, but it does not support concurrent writers and is not a target for a team with parallel pipelines.
- **Full reload only.** `if_exists="replace"` rebuilds the table on every run. There is no incremental load, and therefore no history of how the catalogue changed over time.
- **Console-only output.** Reports print to standard output. Nothing is written to a file or exported for another system to consume.
- **No scheduling.** The pipeline runs when you run it. There is no cron wrapper, no Airflow DAG and no retry policy.
- **Tests duplicate production SQL.** See Section 9 — the queries are verified, but not by importing the module that runs them.
- **No static analysis.** There is no type checking and no linter configuration, so the type of a column is not checked anywhere.

14.2 Roadmap

Next step	What it would deliver
Move the report SQL into shared constants	Tests that import the real queries, closing the regression gap
Configurable limit and pagination	The full 194-record catalogue, with the limit set by argument or environment
Export reports to CSV and Markdown	Output that other tools and people can consume directly
Incremental loads plus a history table	Trend analysis: how stock levels and pricing move between runs
A rejection threshold and warning	An alert when the rejection rate rises, signalling an upstream contract change
Linting, type hints and CI	`ruff`, type annotations, and a GitHub Actions workflow running the suite on every push
Move to PostgreSQL	Concurrent writes, a shared server, and the option of multiple pipelines
A second data source	A comparative pipeline — for example, the same catalogue at two price points

A Command Reference

Command	Purpose
`python-msrc.pipeline`	Run the full four-stage pipeline
`python-msrc.ingestion.fetch_products`	Stage 1 — fetch from the API
`python-msrc.transformation.clean_products`	Stage 2 — validate and split
`python-msrc.database.load_products`	Stage 3 — load into SQLite
`python-msrc.analytics.product_report`	Stage 4 — run the three reports
`python-mpytesttests/-v`	Run the test suite verbosely
`pipinstall-rrequirements.txt`	Install dependencies
`source.venv/bin/activate`	Activate the virtual environment
`sqlite3data/ecommerce.db"SELECT\protect\TU\textellipsis"`	Query the loaded data

B References

- Project repository — `git@github.com:Lebo-Moropene/sa-ecommerce-data-pipeline.git`
- Data source — `https://dummyjson.com/products` (public sample REST API; fetched with `limit=10`, 5s connect and 15s read timeouts)
- Python — `https://docs.python.org/3/`
- `requests` — `https://requests.readthedocs.io/`
- `pandas` — `https://pandas.pydata.org/docs/`

- SQLite — `https://www.sqlite.org/docs.html`
- `pytest` — `https://docs.pytest.org/`

End of document — SA E-Commerce Data Pipeline project documentation.